ECE 5740 Project: Circuit Optimization with ODC's

I. Team Overview

Group Members: Bobby Lofgren, Carson Ord, Tyler Allison

II. Abstract

Our initial topic was Synthesis of Rectification Patches for Buggy Circuits, or for Engineering Change Orders using Partial Logic Synthesis. After discussion and feedback, we examined the problem of optimizing circuits with partial logic synthesis. We will be working with basic circuits, taking circuits, and looping through the nets to optimize and make them more efficient. We wanted to find the lowest-cost solution possible and change the smallest subset of the circuit [1]. Our project was geared more towards experimenting with existing tools rather than designing new ones.

Our main goals are as follows: use ABC to determine ODCs of a net, and take those ODCs and use them to optimize the circuit in SIS.

III. Introduction

Our project is based on many existing topics. This is more geared towards the rectification side, but Craig interpolation is very important. In mathematical terms, Craig Interpolation states that if a formula A implies a formula B is unsatisfiable when combined with their negations in partitioned form, then there exists an interpolant C such that A implies C and C implies B, where C contains only the common variables of A and B.

In Boolean terms, a function F with variables a, b, and c can be broken into subsets {Fa, Fb, Fc}. If F1(Fa, Fc) AND F2(Fb, Fc) have no common satisfying assignments (i.e., are unsatisfiable together), then an interpolation exists.

In circuit rectification, the smallest existing interpolant can be interpreted as the patch of a function [7].

We are working with optimization, so we used other tools to help our circuit. Our main topic of need is ODC's. An ODC (Observability Don't Care) is a condition for which a node's value does not affect the primary outputs of the circuit.

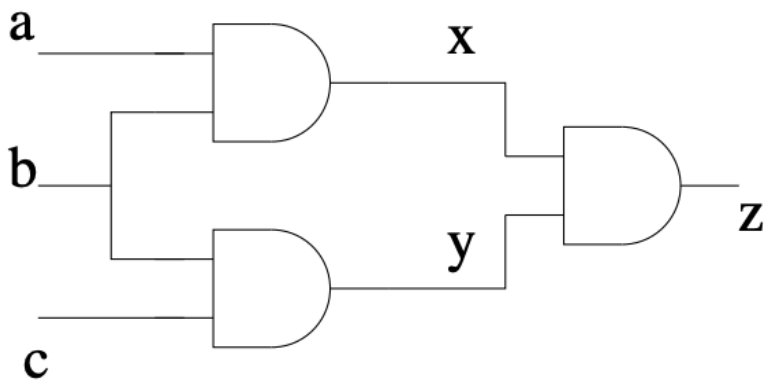

Figure 1. Basic Circuit [6]

In Figure 1, the output Z doesn't care what X is if Y is 0. If Y has the value of 0, Z will always be 0, no matter what. This means the ODCs of X are Y' or (B OR C).

Using ODCs, we can optimize circuits. Our project created miters to compute ODC sets, took these sets to optimize the circuit, and then repeated the process for the next net.

IV. Our Experiment

As mentioned above, we want to compute ODCs to optimize a circuit on all nets of a circuit. ABC has commands to compute miters, which are used to find these don't cares, but just calling commands didn't seem hard enough or enough work to do for the final project.

The bulk of what we did besides testing was using internal ABC commands to compute these ODC sets. Our ABC program takes a BLIF file, finds the ODCs, and adds them in an .exdc block to a new BLIF file. We then give the BLIF file to SIS to optimize.

We use SIS for optimization because SIS preserves and propagates EXDC constraints differently than ABC in our flow, allowing us to explicitly apply computed don't cares during optimization.

Once the circuit was optimized, we moved on to the next net. We compute the next net's ODCs on the updated circuit because structural changes may affect observability conditions. If we didn't, the final circuit might not remain equivalent to the progressively optimized version.

In Figure 1, the ODC of X is Y', and the ODC of Y is X', but if we made both of those calculations independently, then intermediate circuit changes could affect later ODC computation. That is why we recompute ODCs on the updated circuit.

Once we looped through all the nets, our circuit was optimized.

We tested: small, medium, and large circuits; circuits with no ODCs; different traversal orders/ The first traversal was normal ordering (starting at the first net to the last). The other was reverse ordering. To run these loops, we used scripts. We could have done all commands manually, but to simplify execution and results, we wrote scripts so that one command ran the whole loop. Due to SIS limitations, the script outputs the exact commands required for SIS execution per circuit.

V. Code

Our ABC code consisted of two main things: the ODC calculation and file writing. When learning more about the ABC interface, we realized that it uses AIGs (And Inverted Graphs). To make our circuit work without using basic commands, we had to turn it into an AIG so we could use the functions in ABC.

```
// Make a new AIG to hold the Miter with plenty of space for universe 0 and 1
Aig_Man_t* pMiterAig = Aig_ManStart(Aig_ManNodeNum(pAig) * 2);

// Maps of when circuit target = 0 and when target = 1
int arr0Size = Aig_ManObjNumMax(pAig);
int arr1Size = Aig_ManObjNumMax(pAig);
std::vector<Aig_Obj_t*> map0(arr0Size, nullptr); // target = 0
std::vector<Aig_Obj_t*> map1(arr1Size, nullptr); // target = 1

// Map constants between old and new nodes
map0[Aig_ManConst1(pAig)->Id] =  Aig_ManConst1(pMiterAig);
map1[Aig_ManConst1(pAig)->Id] =  Aig_ManConst1(pMiterAig);

Aig_Obj_t* pObj;
int i;

// Loop through all combinational inputs
Aig_ManForEachCi(pAig, pObj, i)
{
    Aig_Obj_t* pNewCi = Aig_ObjCreateCi(pMiterAig);

    // Update maps and set new input pins
    map0[pObj->Id] = pNewCi;
    map1[pObj->Id] = pNewCi;
}
```

Figure 2. AIG Creation

The code in Figure 2 creates two new, separate universes that are available to be worked with. These will be crucial in the creation of a miter. After these two universes are created, the first step is to loop through and map the primary inputs in each.

```
// Force target wire to 0 and 1 in each map
map0[pTargetWire->Id] = Aig_Not(Aig_ManConst1(pMiterAig));
map1[pTargetWire->Id] = Aig_ManConst1(pMiterAig);
```

Figure 3. Cofactoring

Following this, a distinction must be made between the two universes, as up to this point, the two are identical. By intentionally severing the target net in each and forcing it to ground (0) in one and power (1) in the other, this creates the necessary difference, creating negative and positive cofactors. The code in Figure 3 demonstrates this cofactoring.

```
// Loop through all AND gates
Aig_ManForEachNode(pAig, pObj, i)
{
    // Skip target wire
    if (pObj == pTargetWire)
        continue;

    // Find pointers for 0 and 1 children for universe 0
    Aig_Obj_t* pChild0_0 = Aig_NotCond(map0[Aig_ObjFanin0(pObj)->Id], Aig_ObjFaninC0(pObj));
    Aig_Obj_t* pChild1_0 = Aig_NotCond(map0[Aig_ObjFanin1(pObj)->Id], Aig_ObjFaninC1(pObj));
    map0[pObj->Id] = Aig_And(pMiterAig, pChild0_0, pChild1_0);

    // Find pointers for 0 and 1 children for universe 1
    Aig_Obj_t* pChild0_1 = Aig_NotCond(map1[Aig_ObjFanin0(pObj)->Id], Aig_ObjFaninC0(pObj));
    Aig_Obj_t* pChild1_1 = Aig_NotCond(map1[Aig_ObjFanin1(pObj)->Id], Aig_ObjFaninC1(pObj));
    map1[pObj->Id] = Aig_And(pMiterAig, pChild0_1, pChild1_1);
}
```

Figure 4. Duplication

Now that the two universes diverge at a point, the rest of the circuit must be routinely and safely constructed. The code in Figure 4 demonstrates this construction by using the Aig_ManForEachNode() function. The original circuit is traversed through every internal AND gate from input to output, reconstructing two parallel copies of the circuit based on the earlier split.

```
Aig_Obj_t* pCareSet = Aig_Not(Aig_ManConst1(pMiterAig)); // Set care set to 0
Aig_ManForEachCo(pAig, pObj, i)
{
    // Find memory addresses for final gates of universe 0 and 1
    Aig_Obj_t* pOut0 = Aig_NotCond(map0[Aig_ObjFanin0(pObj)->Id], Aig_ObjFaninC0(pObj));
    Aig_Obj_t* pOut1 = Aig_NotCond(map1[Aig_ObjFanin0(pObj)->Id], Aig_ObjFaninC0(pObj));

    // Create Miter
    Aig_Obj_t* pMiterXor = Aig_Exor(pMiterAig, pOut0, pOut1);

    // Accumulate the care sets
    pCareSet = Aig_Or(pMiterAig, pCareSet, pMiterXor);
}
```

Figure 5. Miter Construction

With both universes now entirely filled out, the next step is to create the miter between them. Figure 5 shows the code that performs this action. The final outputs of both universes are taken, and a XOR gate is wired between them. The output of each XOR gate is accumulated into a Care Set, ensuring that the final Care Set will output a 1.

```
// ODC is inverse of care set
Aig_Obj_t* pODC = Aig_Not(pCareSet);
Aig_ObjCreateCo(pMiterAig, pODC);

// Cleanup
map0.clear();
map1.clear();

return pMiterAig;
```

Figure 6. Inversion

The final step of the process is to take the Care Set and invert it to yield the Observability Don't Care logic, then designated as the output of the Miter AIG and returned for synthesis. Figure 6 shows this simple inversion and returning process.

```
// Loops over and gets all used variable names
Vec_Ptr_t* vPiNames = Vec_PtrAlloc(Aig_ManCiNum(pAig));

Abc_NtkForEachCi(pNtk, pPi, i) {
    Vec_PtrPush(vPiNames, Abc_ObjName(pPi));
}

// Gets the output names for the file
Vec_Ptr_t* vPoNames = Vec_PtrAlloc(1);
Vec_PtrPush(vPoNames, (void*)"odc");

// Puts these names and the miter into a new blif
Aig_ManDumpBlif(pMiterAig, "odc_raw.blif", vPiNames, vPoNames);

// Code to add ODC to blif file
std::ifstream orig("func.blif");
std::ifstream odc("odc_raw.blif");
std::ofstream out("func_with_exdc.blif");
```

Figure 7. Making the ODC file

The main function of the program is able to call the miter creation function, then use its output to aid in writing to a new file. The first thing that had to be done was to collect the original input and output names. If our code added incorrect names for inputs and outputs to the exdc section, SIS wouldn't be able to optimize it. We then added the ODC set to its own BLIF file so that we could copy it into the next set.

```
// Copy original blif file up to .end
while (std::getline(orig, line))
{
    if (line == ".end")
        break;

    out << line << "\n";
}

// Start the exdc block
out << "\n.exdc\n";
out << ".inputs a b c\n";
out << ".outputs odc\n";


// Add the odc logic computed in this file
bool copy = false;

while (std::getline(odc, line))
{
    // start copying after first .names
    if (line.rfind(s: ".names", pos: 0) == 0)
        copy = true;

    // stop at end
    if (line == ".end")
        break;

    // skip headers (.inputs/.outputs)
    if (copy && line[0] != '.')
        out << line << "\n";
}
```

Figure 8. BLIF File Creation

Figure 8 shows how we made our new BLIF file. We take the original BLIF and copy it until the end line. We then add in the exdc header with the variable names that we calculated. Finally, we add the code from the ODC file made above to form a complete BLIF file. With this file, now including an exdc block, we can send it to sis to be optimized. We run all of this code on every net to optimize the entire circuit.

VI. Testing

We wanted to test all types of BLIF files and circuits to make sure our code worked as intended. We also wanted to test one other thing. The order of looping through nets. Our first test was running our code on BLIF files that had no ODCs. It correctly found that and didn't write anything new into our BLIF file. Our next test was something simple.

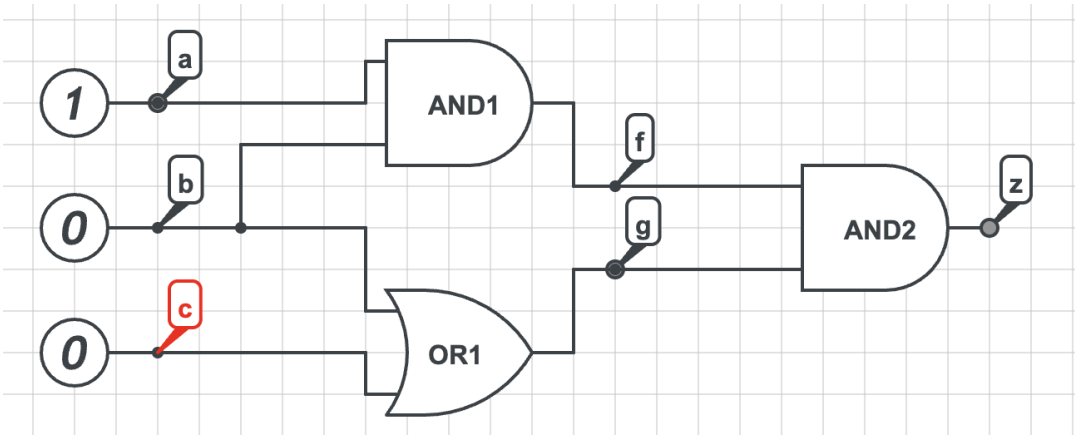

Figure 9. Test BLIF file

This BLIF file does have ODCs. f and g both have ODCs, respectively. When we run our optimizer on net f, it calculates the ODC b'c'. That is correct, and then we simplify the circuit in sis. Once that simplification happens, we can't simplify g's ODCs. Our optimizer notices that and simplifying f is the only change we make. It can limit Z to be equal to ab. If a and b are not both true, the circuit can't be true. If they are, then the circuit is always true. We then ran tests on medium to large circuits. We wanted to optimize them as much as possible, to get as efficient a circuit as possible. Our optimizer can calculate all ODC's for a circuit, and then sis can completely simplify it. Figure 6 shows a larger, more complicated circuit, but using ODCs, it simplifies it down to Z = ab once again.

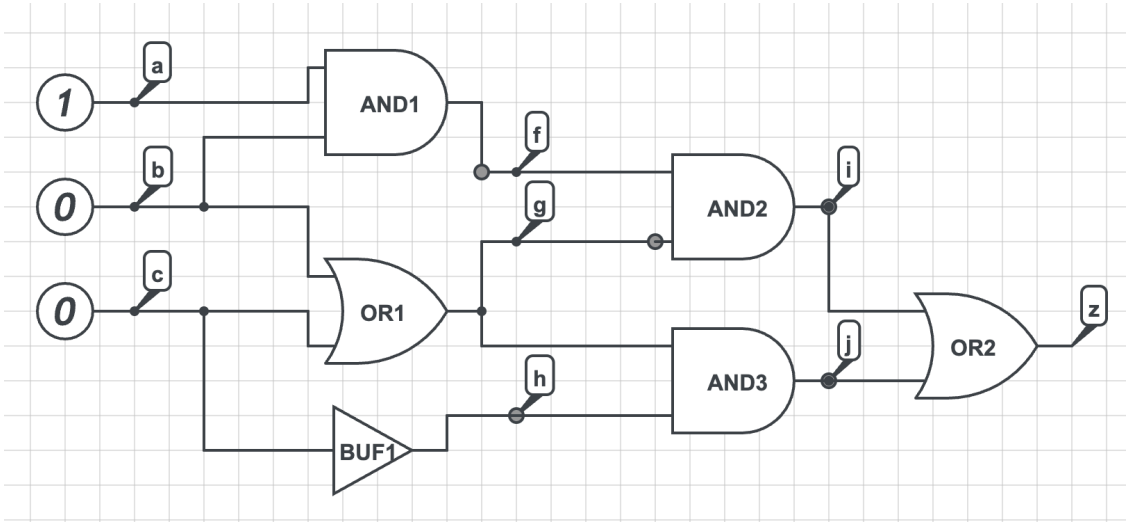

Figure 10. Larger test BLIF file

We ran our tests on many BLIF files to test robustness. We tested on many different sizes, with lots of different combinations of ODCs. Some example results collected for Figures 5 and 6 are: Figure 9: 4 literals down to 2 and 3 gates down to 1, Figure 10: 5 literals down to 2 and 6 gates down to 1. Our code works very well, and the results show that we are optimizing the circuits with their ODCs.

The important idea we ran into is that it is very trivial where we “center” our synthesis around. In the paper, it mentions that the choice of the net is very important and can change the output of the circuit. Calculating ODCs relies on what we see as our main node. The difference in the main node can lead to many different ODCs. This choice is not something that these tools can do, and we had to figure out how to do that ourselves.

VII. Individual Contribution

While we worked on the entire project together, each member focused on one part. The other members would always help them, but this way we were able to evenly divide the project. We all came up with the project plan and implementation ideas together. Below is what each member focused on.

Carson: Set up the project toolchain. Also wrote the initial code to calculate the ODCs in ABC. Carson was able to dive into the ABC specifics thanks to previous course knowledge and use its tools and functions to calculate the ODCs for a specific net.

Bobby: Set up the file writing. Bobby took Carson's ODCs and the original BLIF file and wrote the code to map them all into one file. There was also some debugging that had to be done in the ODC calculation. He also wrote much of the final report.

Tyler: Set up the user-side functionality, created blif test files, and tested the implementation, fixing minor bugs along the way. Discovered the more fundamental problem with our signal selection heuristics and worked on revisions to implement simple windowing and optimize as-you-go simplification.

IX. Conclusion

This project taught us a lot about the effectiveness of ABC and SIS and how ODCs can be used to simplify circuits. We spent most of our time working with ABC’s tools to generate these ODCs. Once generated, we were able to give them to SIS to be optimized, and the circuit was usually significantly more cost-effective. The README in our repository has directions on how to run and work our code. We wanted to make it as easy as possible for users, so we have

steps and outputs that walk you through it. We left example BLIF files in the repository, so that they can be tested with what is in there.

Unfortunately, due to factors such as time and availability, our project is not as complete as we would like for this submission. We were able to make very small optimizations, but only on specific edge case circuits. We couldn't completely "loop" over nets as our ABC tools found ODCs for all variables in a circuit. We spent a lot of time writing code to compute these miters and ODCs because we did not want to use the basic ABC commands. Our main goal was to work with new tools and try to use things that we have never used before.

After recognizing the shortcomings of our core implementation, we worked on changing the project to prevent SIS from optimizing out the necessary cubes and provide useful simplifications. We settled on changing the ODC computation to only use the fanins for the selected net(s) and optimizing with SIS as soon as those ODCs were generated. To create more substantial simplifications, we would then move on to implement a simplified version of windowing [8]. These changes were not fully realized, however, and the WIP version can be viewed on the experimental branch of our GitHub project. Much of this code was hastily written with a lot of refactoring using AI tools, and while this version is in a more complete state than our current build, we don't want to stand behind work we don't fully understand.

X. References

[1] P. Kalla, “Information and Guidance for Class Projects, ECE/CS 6740/5740: CAD of Digital Circuits,” University of Utah, Mar. 2026.

[2] K. F. Tang, C. A. Wu, P. K. Huang, and C. Y. Huang, “Interpolation-Based Incremental ECO Synthesis for Multi-Error Logic Rectification,” in Proc. Design Automation Conf. (DAC), 2011, pp. 146–151.

[3] S. Jo, T. Matsumoto, and M. Fujita, “SAT-Based Automatic Rectification and Debugging of Combinational Circuits with LUT Insertions,” in IEEE 21st Asian Test Symposium (ATS), 2012.

[4] M. Fujita, “Toward Unification of Synthesis and Verification in Topologically Constrained Logic Design,” Proceedings of the IEEE, 2015.

[5] A. Dao, N.-Z. Lee, L.-C. Chen, M. Lin, J.-H. Jiang, A. Mishchenko, and R. Brayton, “Efficient Computations of ECO Patch Functions,” in Proc. Design Auto. Conf., 2018.

[6] Priyank Kalla. *Don’t Cares in Logic Optimization: Definitions and Computations*. University of Utah, ECE/CS 5740/6740: CAD of Digital Circuits

[7] Kalla, Priyank. *Partial Logic Synthesis: Synthesis of ECO & Rectification Functions*. University of Utah, ECE/CS 5740/6740: CAD of Digital Circuits

[8] A. Mischenko, R. K. Brayton, J. H. R. Jiang, and J. Stephen, “Scalable Don’t-care-based Logic Optimization and Resynthesis,” ACM Trans. Reconfigurable Technol. Syst., 2011.